GNU Hurd Filesystem Development Guide

In the GNU Hurd, a file system is not part of the kernel. Instead, it is written as a **user-space server called a translator**. Because the Hurd is built on top of the Mach microkernel, a file system is simply a normal program that processes Inter-Process Communication (RPC) requests from clients (via `glibc`) and translates them into file operations.

Official & Community Documentation

Developing a Hurd translator requires getting comfortable with the architecture's foundational libraries.

1. **The GNU Hurd Reference Manual**: This is the primary authority. Refer specifically to Chapter 4 (IO and Pager Interfaces) and Chapter 5 (Files, Translators, and helper libraries).
2. **The GNU Hurd Wiki - Translator Hub**: The core starting page for understanding how translators function, how to debug them, and where to find source code.
3. **The Hurd Hacking Guide**: Contains tutorials for writing simple translators.

The 3 Core Libraries for Filesystems

You do not need to implement raw Mach RPC structures from scratch. The Hurd provides three dedicated C libraries tailored to different types of file systems:

- **`libdiskfs`**: Physical filesystems that back out to a physical disk or block device. Examples: `ext2fs`, `fatfs`, `iso9660fs`.
- **`libnetfs`**: Networked or highly virtualized file systems that handle their own nodes in memory. Examples: `nfs`, `ftpfs`, `9pfs`.
- **`trivfs`**: Simple, single-node translators that do not present a directory structure. Examples: `/dev/null`, `/dev/zero`, pipes.

Step-by-Step Development Workflow

1. Choose Your Library & Implement Callbacks

When writing a file system, your program implements structural callback functions defined by the library (`libdiskfs` or `libnetfs`). For instance, you will override functions to: * Look up a node (`diskfs_lookup`) * Read node data (`diskfs_node_rdwr`) * Write to disk (`diskfs_write_disknode`)

2. Initialize the Main Loop

Your `main()` function handles setting up the store (the underlying raw media or socket connection) and hands execution off to the Hurd's ports management architecture:

```
#include <version.h>
#include <hurd/diskfs.h>

int main (int argc, char **argv) {
    /* 1. Parse command line options using argp */
    /* 2. Open your underlying storage media */
    /* 3. Initialize diskfs structures */

    // Begin the server loop to wait for Mach RPC requests
    ports_manage_port_operations_one_thread (diskfs_port_bucket,
                                             diskfs_demuxer,
                                             0);
    return 0;
}
```

3. Test and Attach the Translator via `settrans`

Because Hurd filesystems are just userland processes, you don't risk kernel panics while debugging. Compile your executable (e.g., `myfs`) and attach it to a directory node using the standard `settrans` command:

```
# Create an empty node to act as the mount point
touch /tmp/my_mount_point

# Set your compiled translator passively onto that node
settrans -c /tmp/my_mount_point ./myfs /path/to/disk.img
```

The next time any application runs `ls` or tries to access `/tmp/my_mount_point`, the underlying filesystem will transparently wake your `myfs` binary up to service the request.

Best Advice for Beginners

Read the existing source code. The easiest way to learn is by looking at simple, production-ready code inside the official Hurd tree. Run `apt source hurd` on a Debian GNU/Hurd virtual machine and inspect: * `hurd/trans/` (Contains simple `trivfs` projects like `streamio` or `null`) * `hurd/fatfs/` (A readable, standard `libdiskfs` example)